

# Conception et Optimisation d'une Base de Données de Gestion de la Recherche

TP1 - Gestion de Bases de Données et Data Warehouse

Master 1 Data Engineering (M1 DENG)

Université de Corse

Réalisé par :

Nom Prénom 1

Nom Prénom 2

Nom Prénom 3

Date de rendu : 12 novembre 2025

# Table des matières

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                      | <b>4</b>  |
| 1.1      | Contexte du projet . . . . .                             | 4         |
| 1.2      | Objectifs . . . . .                                      | 4         |
| 1.3      | Organisation du rapport . . . . .                        | 4         |
| <b>2</b> | <b>Partie 1 - Conception de la Base de Données</b>       | <b>5</b>  |
| 2.1      | Méthodologie et utilisation de l'IA générative . . . . . | 5         |
| 2.1.1    | Outils utilisés . . . . .                                | 5         |
| 2.1.2    | Démarche adoptée . . . . .                               | 5         |
| 2.1.3    | Résultats . . . . .                                      | 5         |
| 2.1.4    | Analyse critique . . . . .                               | 5         |
| 2.2      | Interprétation du cahier des charges . . . . .           | 6         |
| 2.2.1    | Entités principales identifiées . . . . .                | 6         |
| 2.2.2    | Relations entre entités . . . . .                        | 6         |
| 2.2.3    | Hypothèses et choix de conception . . . . .              | 6         |
| 2.2.4    | Dictionnaire des attributs . . . . .                     | 7         |
| 2.3      | Schéma relationnel . . . . .                             | 10        |
| 2.3.1    | Vue d'ensemble . . . . .                                 | 10        |
| 2.3.2    | Contraintes CHECK . . . . .                              | 11        |
| <b>3</b> | <b>Partie 2 - Peuplement de la Base de Données</b>       | <b>12</b> |
| 3.1      | Méthodologie . . . . .                                   | 12        |
| 3.1.1    | Outils utilisés . . . . .                                | 12        |
| 3.1.2    | Démarche générale . . . . .                              | 12        |
| 3.2      | Architecture du script Python . . . . .                  | 12        |
| 3.2.1    | Structure du code . . . . .                              | 12        |
| 3.2.2    | Gestion de l'intégrité référentielle . . . . .           | 13        |
| 3.3      | Génération des données par table . . . . .               | 13        |
| 3.3.1    | Institutions . . . . .                                   | 13        |
| 3.3.2    | Laboratoires . . . . .                                   | 13        |
| 3.3.3    | Chercheurs . . . . .                                     | 14        |
| 3.3.4    | Projets . . . . .                                        | 14        |
| 3.3.5    | Contrats . . . . .                                       | 14        |
| 3.3.6    | Publications . . . . .                                   | 14        |
| 3.3.7    | Jeux de données . . . . .                                | 15        |
| 3.4      | Cohérence des données générées . . . . .                 | 15        |
| 3.4.1    | Statistiques des données générées . . . . .              | 15        |
| 3.5      | Exemples de code . . . . .                               | 16        |
| 3.5.1    | Génération des projets . . . . .                         | 16        |
| 3.5.2    | Génération des chercheurs . . . . .                      | 16        |

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| <b>4</b> | <b>Partie 3 - Gestion des Utilisateurs et Création de Vues</b> | <b>18</b> |
| 4.1      | Stratégie de sécurité                                          | 18        |
| 4.1.1    | Objectifs                                                      | 18        |
| 4.1.2    | Architecture des rôles                                         | 18        |
| 4.2      | Définition des rôles                                           | 18        |
| 4.2.1    | Rôle Chercheur                                                 | 18        |
| 4.2.2    | Rôle Data Manager                                              | 18        |
| 4.2.3    | Rôle Administrateur                                            | 18        |
| 4.3      | Vues créées                                                    | 18        |
| 4.3.1    | Vue 1 : Mes_Projets                                            | 18        |
| 4.3.2    | Vue 2 : Metadata_Jeux_Donnees                                  | 19        |
| 4.3.3    | Vue 3 : Publications_Par_Laboratoire                           | 19        |
| 4.3.4    | Vue 4 : Contrats_Actifs                                        | 19        |
| 4.3.5    | Vue 5 : Projets_Budget_Global                                  | 19        |
| 4.4      | Attribution des privilèges                                     | 19        |
| 4.4.1    | Privilèges par rôle                                            | 19        |
| 4.4.2    | Exemples de scripts SQL                                        | 20        |
| <b>5</b> | <b>Partie 4 - Requêtes et Optimisation</b>                     | <b>21</b> |
| 5.1      | Méthodologie d'optimisation                                    | 21        |
| 5.1.1    | Démarche                                                       | 21        |
| 5.2      | Requête R1 - Jeux de données par projet                        | 21        |
| 5.2.1    | Énoncé                                                         | 21        |
| 5.2.2    | Version 1 : Utilisation d'une sous-requête avec WITH           | 21        |
| 5.2.3    | Version 2 : Jointures                                          | 21        |
| 5.2.4    | Analyse des performances                                       | 22        |
| 5.2.5    | Plans d'exécution                                              | 23        |
| 5.2.6    | Optimisation choisie                                           | 25        |
| 5.3      | Requête R2 - Projets sans chercheurs peu productifs            | 25        |
| 5.3.1    | Énoncé                                                         | 25        |
| 5.3.2    | Version 1 : Utilisation de NOT IN                              | 25        |
| 5.3.3    | Version 2 : Utilisation de EXCEPT                              | 26        |
| 5.3.4    | Analyse des performances                                       | 27        |
| 5.3.5    | Plans d'exécution                                              | 27        |
| 5.3.6    | Optimisation choisie                                           | 28        |
| 5.4      | Requête R3 - Laboratoires conformes                            | 29        |
| 5.4.1    | Énoncé                                                         | 29        |
| 5.4.2    | Version 1 : Utilisation de NOT EXISTS                          | 29        |
| 5.4.3    | Version 2 : Utilisation de LEFT JOIN                           | 29        |
| 5.4.4    | Analyse des performances                                       | 30        |
| 5.4.5    | Plans d'exécution                                              | 30        |
| 5.4.6    | Optimisation choisie                                           | 31        |
| 5.5      | Synthèse des optimisations                                     | 33        |
| 5.5.1    | Index créés                                                    | 33        |

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>6</b> | <b>Partie 5 - Triggers et Procédures Stockées</b> | <b>35</b> |
| 6.1      | Triggers                                          | 35        |
| 6.1.1    | Trigger 1 : Limite de participants à un projet    | 35        |
| 6.1.2    | Trigger 2 : Vérification DMP d'un jeu de données  | 35        |
| 6.2      | Procédures stockées et fonctions                  | 35        |
| 6.2.1    | Fonction 1 : Nombre de publications d'un projet   | 35        |
| 6.2.2    | Procédure 2 : Préparation du bilan d'un projet    | 36        |
| 6.2.3    | Fonction 3 : Fiche projet                         | 36        |
| 6.2.4    | Procédure 4 : Archivage des contrats échus        | 36        |
| 6.3      | Documentation du code                             | 37        |
| 6.3.1    | Conventions de nommage                            | 37        |
| 6.3.2    | Commentaires                                      | 37        |
| <b>7</b> | <b>Conclusion</b>                                 | <b>38</b> |
| 7.1      | Synthèse du travail réalisé                       | 38        |
| 7.2      | Apports pédagogiques                              | 38        |
| 7.2.1    | Compétences techniques acquises                   | 38        |
| 7.2.2    | Utilisation de l'IA générative                    | 38        |
| 7.3      | Difficultés rencontrées                           | 38        |
| 7.4      | Perspectives d'amélioration                       | 38        |
| <b>A</b> | <b>Scripts SQL</b>                                | <b>39</b> |
| A.1      | Script de création du schéma                      | 39        |
| A.2      | Script d'insertion des données                    | 39        |
| A.3      | Script de gestion des utilisateurs                | 39        |
| A.4      | Scripts des requêtes optimisées                   | 39        |
| A.5      | Scripts des triggers et procédures                | 39        |
| <b>B</b> | <b>Code Python</b>                                | <b>39</b> |
| B.1      | Script de génération de données                   | 39        |
| <b>C</b> | <b>Captures d'écran</b>                           | <b>39</b> |
| C.1      | Plans d'exécution                                 | 39        |
| C.2      | Tests des procédures                              | 39        |
| <b>D</b> | <b>Dictionnaire de données</b>                    | <b>39</b> |

# 1. Introduction

## 1.1. Contexte du projet

Ce projet s'inscrit dans le cadre du TP1 du module de Gestion de Bases de Données et Data Warehouse. L'objectif est de concevoir et implémenter une base de données relationnelle pour la gestion des données de recherche d'une université.

## 1.2. Objectifs

Les objectifs principaux, comme énoncés dans le sujet, sont :

- Concevoir un schéma relationnel cohérent pour gérer les institutions, laboratoires, projets, chercheurs, contrats, publications et jeux de données
- Peupler la base avec des données fictives réalistes
- Implémenter une gestion sécurisée des accès via des rôles et des vues
- Optimiser les requêtes complexes
- Mettre en œuvre des triggers et procédures stockées

Ces objectifs seront développés dans les différentes parties de ce rapport.

## 1.3. Organisation du rapport

Ce rapport présente successivement :

- La conception de la base de données (Partie 1) qui s'appuie sur l'utilisation d'outils d'IA générative.
- Le peuplement de la base (Partie 2)
- La gestion des utilisateurs et des vues (Partie 3)
- L'optimisation des requêtes (Partie 4)
- Les triggers et procédures stockées (Partie 5)

## 2. Partie 1 - Conception de la Base de Données

### 2.1. Méthodologie et utilisation de l'IA générative

#### 2.1.1 Outils utilisés

La phase de conception a été assistée par l'agent de code Claude Sonnet 4.5 par le biais de l'extension Visual Studio Code "GitHub Copilot Chat".

Cet outil d'IA générative a principalement permis, à partir d'instructions et du contexte de l'environnement de développement, de proposer des suggestions de code, des structures de données, et d'aider à la modélisation conceptuelle, le tout par le biais de création de fichiers de façon automatique.

#### 2.1.2 Démarche adoptée

1. Analyse du cahier des charges
2. Identification des entités et relations
3. Validation et raffinement avec l'IA
4. Analyse critique des propositions et corrections

#### 2.1.3 Résultats

L'IA a permis de générer une première version du schéma relationnel, incluant les tables, attributs, types de données et contraintes d'intégrité.

Après lui avoir donné le contexte de l'exercice et les exigences (par un simple copier-coller du cahier des charges), nous avons obtenu :

- Un schéma relationnel complet
- Des descriptions détaillées des tables
- Des suggestions de contraintes d'intégrité
- Les requêtes SQL de création
- Un fichier .dbml pour visualiser le MCD
- Des règles métier et triggers de validation (que nous avons supprimés par la suite car non demandés, et non conformes au sujet du TP)

#### 2.1.4 Analyse critique

L'utilisation de l'IA générative nous a été extrêmement utile pour obtenir une base solide, cohérente dans un temps très réduit. Les suggestions étaient globalement pertinentes et conformes au cahier des charges.

Nous avons rencontré cependant une limitation qui ne nous a pas réellement pénalisés : l'IA a pris volontairement la décision de simplifier la relation entre deux entités (Chercheur et Projet) en optant pour une relation 1-N au lieu de N-N (un chercheur ne pouvait être que dans un seul projet.. Or ce n'est pas réaliste et assez contraignant), ce qui n'était pas conforme au sujet. Nous avons corrigé cela en y ajoutant une table d'association.

## 2.2. Interprétation du cahier des charges

### 2.2.1 Entités principales identifiées

- Institution
- Laboratoire
- Projet
- Chercheur
- Contrat
- Publication
- Jeu de données

### 2.2.2 Relations entre entités

- **Institution** → **Laboratoire** : Une institution héberge un ou plusieurs laboratoires (1-N). Chaque laboratoire appartient à une seule institution ;
- **Laboratoire** → **Projet** : Un laboratoire pilote un ou plusieurs projets (1-N). Chaque projet a un laboratoire pilote unique ;
- **Laboratoire** → **Chercheur** : Un laboratoire regroupe plusieurs chercheurs (1-N). Chaque chercheur est rattaché à un laboratoire ;
- **Projet** ↔ **Chercheur** : Relation N-N modélisée par la table d'association **projet\_chercheur**. Un projet peut impliquer plusieurs chercheurs, et un chercheur peut participer à plusieurs projets. Cette table permet de gérer le rôle, la période de participation et le pourcentage d'effort de chaque chercheur sur chaque projet ;
- **Projet** → **Chercheur (responsable)** : Un projet a un chercheur responsable (1-1 optionnel). Un chercheur peut être responsable de plusieurs projets ;
- **Projet** → **Contrat** : Un projet peut être financé par plusieurs contrats (1-N). Chaque contrat finance un seul projet ;
- **Contrat** → **Jeu \_Donnees** : Un contrat peut générer plusieurs jeux de données (1-N). Chaque jeu de données est associé à un unique contrat ;
- **Chercheur** → **Jeu \_Donnees** : Un chercheur peut produire plusieurs jeux de données (1-N). Chaque jeu de données a un auteur principal unique ;
- **Publication** ↔ **Chercheur** : Relation N-N modélisée par la table d'association **publication\_auteur**. Une publication peut avoir plusieurs auteurs, et un chercheur peut publier plusieurs articles. L'attribut **ordre\_auteur** permet de gérer l'ordre de signature.

### 2.2.3 Hypothèses et choix de conception

Selon nos connaissances sur le domaine de la recherche académique, nous avons fait les choix suivants :

- Un chercheur peut participer à plusieurs projets simultanément ;
- Un projet peut avoir plusieurs chercheurs responsables (co-responsables) mais pour simplifier, nous avons choisi un seul responsable principal ;
- Les publications ne sont pas directement rattachées aux projets, mais aux chercheurs. Cela reflète mieux la réalité où les chercheurs publient indépendamment de leurs projets ;

- Les jeux de données sont liés aux contrats car ils résultent souvent des financements obtenus ;
- Les types énumérés (ENUM) ont été définis pour les attributs avec des valeurs limitées (type d'institution, type de contrat, statut DMP).

#### 2.2.4 Dictionnaire des attributs

Le dictionnaire nous a permis de clarifier le rôle de chaque table et attribut, ainsi que les contraintes associées.

##### Types énumérés `type_institution` :

- `universite` : Université
- `organisme_recherche` : Organisme de recherche
- `partenaire_privé` : Partenaire privé

##### `type_contrat` :

- `ANR` : Agence Nationale de la Recherche
- `H2020` : Horizon 2020 (programme européen)
- `Region` : Financement régional
- `Europe` : Financement européen
- `Autre` : Autre type de financement

##### `statut_dmp` :

- `brouillon` : DMP en cours de rédaction
- `soumis` : DMP soumis pour validation
- `valide` : DMP validé

**Table Institution** Institution de rattachement (université, organisme de recherche ou partenaire privé).

| Attribut         | Type             | Description                             |
|------------------|------------------|-----------------------------------------|
| id               | bigint           | Identifiant unique (PK, auto-increment) |
| nom              | text             | Nom de l'institution (NOT NULL)         |
| type_institution | type_institution | Type d'institution (NOT NULL, enum)     |
| adresse          | text             | Adresse postale de l'institution        |

TABLE 1 – Attributs de la table Institution. Mise en forme LaTeX par une IA générative.

**Table Laboratoire** Laboratoire de recherche rattaché à une institution.



| Attribut       | Type   | Description                             |
|----------------|--------|-----------------------------------------|
| id             | bigint | Identifiant unique (PK, auto-increment) |
| nom            | text   | Nom du laboratoire (NOT NULL)           |
| id_institution | bigint | FK vers Institution (NOT NULL)          |

TABLE 2 – Attributs de la table Laboratoire. Mise en forme LaTeX par une IA générative.

**Table Projet** Projet de recherche structurant dirigé par un chercheur unique.

| Attribut                 | Type          | Description                        |
|--------------------------|---------------|------------------------------------|
| id                       | bigint        | Identifiant unique (PK)            |
| titre                    | text          | Titre du projet (NOT NULL)         |
| description              | text          | Description détaillée              |
| discipline               | text          | Discipline scientifique (NOT NULL) |
| budget_annuel_eur        | decimal(12,2) | Budget annuel $\geq 0$ (NOT NULL)  |
| date_debut               | date          | Date de début (NOT NULL)           |
| date_fin                 | date          | Date de fin $\geq$ date_debut      |
| id_laboratoire_pilote    | bigint        | FK vers Laboratoire (NOT NULL)     |
| id_chercheur_responsable | bigint        | FK vers Chercheur                  |

TABLE 3 – Attributs de la table Projet. Mise en forme LaTeX par une IA générative.

**Table Chercheur** Chercheur affilié à un laboratoire.

| Attribut       | Type        | Description                            |
|----------------|-------------|----------------------------------------|
| id             | bigint      | Identifiant unique (PK)                |
| prenom         | text        | Prénom (NOT NULL)                      |
| nom            | text        | Nom (NOT NULL)                         |
| email          | text        | Email professionnel (NOT NULL, UNIQUE) |
| orcid          | varchar(19) | Identifiant ORCID (UNIQUE)             |
| discipline     | text        | Discipline de recherche                |
| id_laboratoire | bigint      | FK vers Laboratoire (NOT NULL)         |

TABLE 4 – Attributs de la table Chercheur. Mise en forme LaTeX par une IA générative.

**Table Projet\_Chercheur** Table associative N-N entre projets et chercheurs.

| Attribut     | Type         | Description                          |
|--------------|--------------|--------------------------------------|
| id           | bigint       | Identifiant unique (PK)              |
| id_projet    | bigint       | FK vers Projet (NOT NULL)            |
| id_chercheur | bigint       | FK vers Chercheur (NOT NULL)         |
| role         | text         | Rôle dans le projet                  |
| date_debut   | date         | Date de début de participation       |
| date_fin     | date         | Date de fin de participation         |
| charge_pct   | decimal(5,2) | Pourcentage d'effort (0-100)         |
| is_principal | boolean      | Chercheur principal (défaut : false) |

TABLE 5 – Attributs de la table Projet\_Chercheur. Mise en forme LaTeX par une IA générative.

**Table Contrat** Contrat de financement d'un projet.

| Attribut            | Type          | Description                       |
|---------------------|---------------|-----------------------------------|
| id                  | bigint        | Identifiant unique (PK)           |
| type_contrat        | type_contrat  | Type de contrat (NOT NULL, enum)  |
| financeur           | text          | Organisme financeur (NOT NULL)    |
| intitule            | text          | Intitulé du contrat (NOT NULL)    |
| montant_eur         | decimal(14,2) | Montant total $\geq 0$ (NOT NULL) |
| duree_mois          | int           | Durée en mois ( $> 0$ )           |
| date_debut          | date          | Date de début (NOT NULL)          |
| date_fin            | date          | Date de fin (NOT NULL)            |
| id_projet           | bigint        | FK vers Projet (NOT NULL)         |
| statut_dmp          | statut_dmp    | Statut DMP (NOT NULL)             |
| date_validation_dmp | date          | Date validation DMP               |
| url_document_dmp    | text          | URL du document DMP               |

TABLE 6 – Attributs de la table Contrat. Mise en forme LaTeX par une IA générative.

**Table Publication** Publication scientifique (métadonnées uniquement).

| Attribut         | Type   | Description                        |
|------------------|--------|------------------------------------|
| id               | bigint | Identifiant unique (PK)            |
| titre            | text   | Titre de la publication (NOT NULL) |
| doi              | text   | Digital Object Identifier (UNIQUE) |
| date_publication | date   | Date de publication                |
| nb_pages         | int    | Nombre de pages (> 0)              |
| url_externe      | text   | URL vers le document (ex : HAL)    |

TABLE 7 – Attributs de la table Publication. Mise en forme LaTeX par une IA générative.

**Table Publication\_Auteur** Table associative N-N entre publications et chercheurs.

| Attribut       | Type   | Description                    |
|----------------|--------|--------------------------------|
| id_publication | bigint | FK vers Publication (NOT NULL) |
| id_chercheur   | bigint | FK vers Chercheur (NOT NULL)   |
| ordre_auteur   | int    | Ordre d'auteur (> 0, NOT NULL) |

TABLE 8 – Attributs de la table Publication\_Auteur. Mise en forme LaTeX par une IA générative.

**Table Jeu\_Donnees** Jeu de données produit dans le cadre d'un contrat.

| Attribut         | Type   | Description                        |
|------------------|--------|------------------------------------|
| id               | bigint | Identifiant unique (PK)            |
| id_contrat       | bigint | FK vers Contrat (NOT NULL)         |
| description      | text   | Description du dataset (NOT NULL)  |
| id_auteur        | bigint | FK vers Chercheur (NOT NULL)       |
| conditions_acces | text   | Conditions d'accès                 |
| licence          | text   | Licence d'utilisation              |
| date_creation    | date   | Date de création du jeu de données |
| date_depot       | date   | Date de dépôt officiel             |
| url_externe      | text   | URL vers l'entrepôt                |

TABLE 9 – Attributs de la table Jeu\_Donnees. Mise en forme LaTeX par une IA générative.

## 2.3. Schéma relationnel

### 2.3.1 Vue d'ensemble

Présentation du schéma relationnel complet (diagramme). Voir Figure 1.

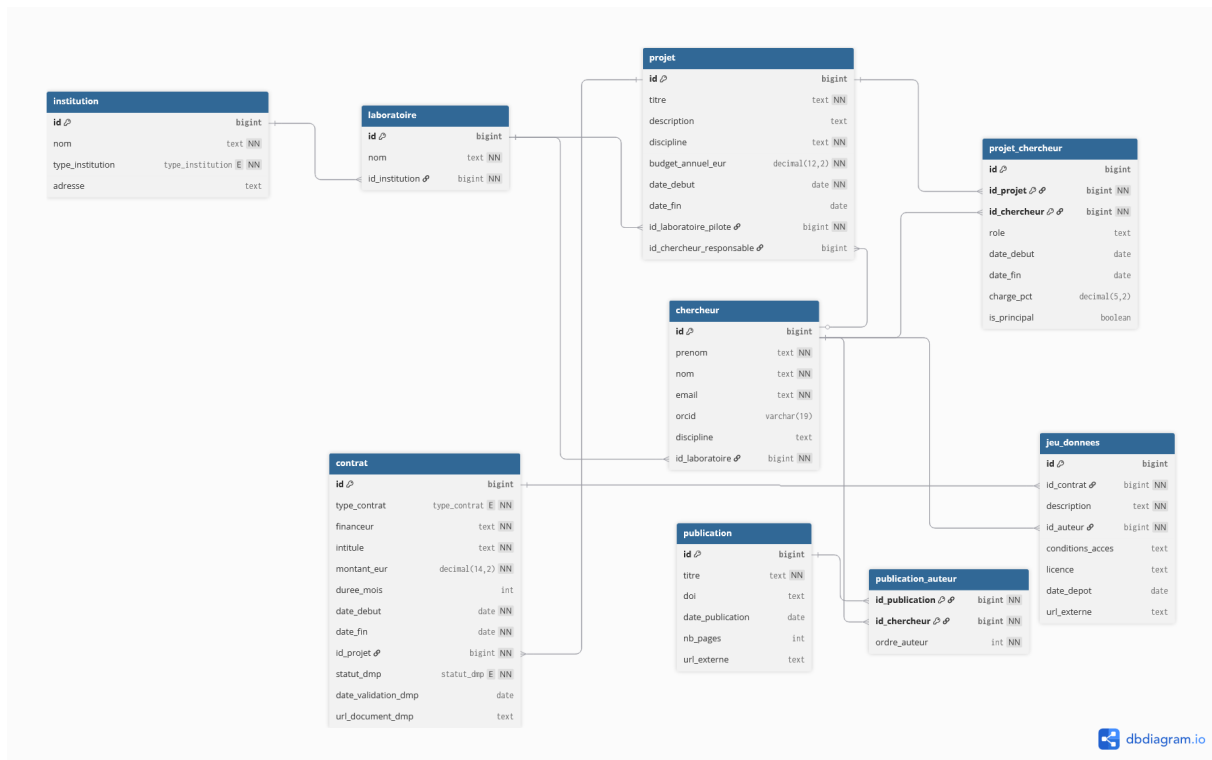


FIGURE 1 – Schéma relationnel de la base de données. Effectué sur [DBDiagram.io](https://dbdiagram.io). En utilisant le langage DBML.

### 2.3.2 Contraintes CHECK

- Vérification que  $\text{budget\_annuel\_eur} \geq 0$
- Vérification que  $\text{montant\_eur} \geq 0$
- Vérification que  $\text{date\_fin} \geq \text{date\_debut}$
- Validation du statut DMP et de ses champs associés

## 3. Partie 2 - Peuplement de la Base de Données

### 3.1. Méthodologie

#### 3.1.1 Outils utilisés

- Python 3.x
- Bibliothèque Faker
- PostgreSQL

#### 3.1.2 Démarche générale

Pour mener à bien ce générateur de données, nous avons tout d'abord analysés les types de données nécessaires pour chaque table, ainsi que les contraintes d'intégrité référentielle.

Dans notre cas, nous avons besoin de générer des emails, des dates cohérentes, des montants financiers, des identifiants uniques, de faux noms et prénoms, des titres de projets et publications, etc.

Pour ce faire, la simple utilisation de la Bibliothèque Faker nous a permis de générer la majorité des données de manière réaliste. Le reste des données spécifiques (comme les types de contrats, le statut DMP, etc.) ont été gérées par des listes prédéfinies.

Ensuite, nous avons structuré le script Python pour générer les données dans un ordre respectant les dépendances entre les tables (par exemple, générer les institutions avant les laboratoires, etc.).

L'analyse des contraintes d'intégrité est la partie la plus délicate, car il faut s'assurer que les clés étrangères pointent vers des enregistrements existants, et que les données respectent les contraintes de dates et autres règles métier. Le tout dans un code maintenable, modulaire et clair, pour faciliter les modifications futures.

### 3.2. Architecture du script Python

#### 3.2.1 Structure du code

Le script Python est organisé en plusieurs fonctions modulaires qui génèrent les données de manière séquentielle en respectant les dépendances :

- **Fonctions utilitaires :**
  - `_quote_sql()` : Échappement sécurisé des valeurs SQL ;
  - `gen_orcid()` : Génération d'identifiants ORCID au format standard ;
  - `random_date_between()` : Génération de dates cohérentes dans un intervalle.
- **Fonction d'en-tête :**
  - `write_header()` : Inclusion du schéma et initialisation de la transaction.
- **Fonctions de génération par table** (dans l'ordre des dépendances) :
  - `generate_institutions()` : Création des institutions avec types variés ;
  - `generate_laboratoires()` : Rattachement aux institutions avec noms uniques ;
  - `generate_chercheurs()` : Génération avec emails et ORCID uniques ;
  - `generate_projets()` : Attribution de responsables et dates cohérentes ;
  - `generate_contrats()` : Gestion des statuts DMP et retour des contrats validés ;

- `generate_jeux_donnees()` : Respect des contraintes DMP (dépôt uniquement si validé) ;
- `generate_publications()` : Création avec DOI uniques et auteurs multiples ;
- `generate_projet_chercheur()` : Table associative N-N avec rôles et charges.
- **Fonction principale :**
  - `main()` : Orchestration de l'ensemble du processus de génération.

Le script utilise un système de retour des identifiants générés pour garantir la cohérence référentielle entre les tables. Chaque fonction retourne la liste des IDs créés, qui sont ensuite utilisés comme clés étrangères par les fonctions suivantes.

### 3.2.2 Gestion de l'intégrité référentielle

Nous avons mis en place plusieurs mécanismes pour garantir l'intégrité référentielle et le respect des différentes contraintes :

- **Propagation des IDs** : Chaque fonction de génération retourne la liste des identifiants créés, qui sont passés en paramètre aux fonctions suivantes
- **Gestion des contraintes d'unicité** :
  - Utilisation de sets Python (`emails_used`, `orcid_ids_used`, `dois_used`) pour éviter les doublons ;
  - Système de tentatives multiples (jusqu'à 100 tentatives pour trouver une valeur unique sinon mise à `null`) pour générer des valeurs uniques ;
  - Mécanisme de fallback avec suffixes numériques en cas d'échec.
- **Respect des contraintes temporelles** :
  - Vérification que `date_fin`  $\geq$  `date_debut` pour les projets et contrats ;
  - Génération de dates de validation DMP postérieures au début du contrat ;
  - Cohérence des dates de participation chercheur-projet.
- **Respect des règles métier** :
  - Dépôt de jeux de données uniquement pour les contrats avec DMP validé ;
  - Attribution de chercheurs responsables existants aux projets ;
  - Génération de noms de laboratoires uniques par institution.
- **Mise à jour des séquences** : Réinitialisation automatique des séquences PostgreSQL après chaque table avec `setval()` pour permettre des insertions manuelles ultérieures.

## 3.3. Génération des données par table

### 3.3.1 Institutions

- **Nombre généré** : Paramétrable, par défaut `max(3,  $\lfloor rows/10 \rfloor$ )` ;
- Génération via `fake.company()` avec types variés (université, organisme de recherche, partenaire privé) ;
- **Exemple** : "Martin Technologies", type "partenaire\_prive", adresse "12 rue de la Paix, 75002 Paris".

### 3.3.2 Laboratoires

- **Nombre généré** : `max(5,  $\lfloor rows/8 \rfloor$ )` ;

- Rattachement d'une institution aléatoire avec garantie d'unicité des noms par institution ;
- **Exemple** : "Deliver innovative Lab", rattaché à l'institution 2.

### 3.3.3 Chercheurs

- **Nombre généré** :  $\max(20, rows)$  (paramètre principal) ;
- **Attributs générés** :
  - Email unique construit à partir du prénom et nom ;
  - ORCID unique généré pour 50% des chercheurs (format XXXX-XXXX-XXXX-XXXX) ;
  - Discipline parmi 5 domaines (Physique, Biologie, Informatique, Chimie, Mathématiques).
- **Cohérence** : Rattachement aléatoire mais cohérent à un laboratoire existant ;
- **Exemple** : Marie Dubois, marie.dubois@example.com, ORCID 1234-5678-9012-3456, discipline Informatique.

### 3.3.4 Projets

- **Nombre généré** :  $\max(10, \lfloor rows/5 \rfloor)$  ;
- Sélection aléatoire d'un chercheur existant pour le chercheur responsable (peut être NULL) ;
- Date de début entre -5 ans et aujourd'hui, date de fin entre date\_debut et +3 ans ;
- Montants réalistes du budget (entre 10 000€ et 500 000€) ;
- **Exemple** : "Optimize cross-platform networks", budget 125 000€, du 2020-03-15 au 2023-12-31.

### 3.3.5 Contrats

- **Nombre généré** :  $\max(10, \lfloor rows/4 \rfloor)$  ;
- Distribution des types réaliste entre ANR, H2020, Région, Europe, Autre ;
- Montants Entre 10 000€ et 2 000 000€ ;
- **Gestion DMP** :
  - Statuts pondérés : 50% brouillon, 30% soumis, 20% validé ;
  - Date de validation et URL uniquement pour les contrats validés ;
  - Retour séparé des contrats validés pour les jeux de données.

### 3.3.6 Publications

- **Nombre généré** :  $\max(10, \lfloor rows/5 \rfloor)$  ;
- Génération unique de DOI pour 40% des publications (format 10.XXXX/XXXXXX) ;
- Entre 1 et 5 auteurs par publication avec ordre de signature ;
- Métadonnées : Date de publication, nombre de pages, URL externe (HAL) ;
- **Exemple** : "Impact of climate on biodiversity", DOI 10.1234/abc123, 3 auteurs, 15 pages.

### 3.3.7 Jeux de données

- **Nombre généré** :  $\max(5, \lfloor \text{rows}/6 \rfloor)$  ;
- Date de dépôt générée uniquement pour les contrats avec DMP validé (40% des cas) ;
- Distribution des Licences entre CC-BY, CC0, ODbL ou NULL ;
- Conditions d'accès entre Ouvert, restreint ou sur demande ;
- **Exemple** : Dataset associé au contrat validé 15, licence CC-BY, déposé le 2024-05-12.

## 3.4. Cohérence des données générées

Les données générées respectent plusieurs niveaux de cohérence :

1. **Cohérence temporelle** : Toutes les dates respectent les contraintes CHECK ( $\text{date\_fin} \geq \text{date\_debut}$ )
2. **Cohérence référentielle** : Toutes les clés étrangères pointent vers des enregistrements existants grâce au système de propagation des IDs
3. **Cohérence métier** :
  - Les jeux de données avec `date_depot` sont liés à des contrats DMP validés
  - Les chercheurs responsables de projets appartiennent à des laboratoires existants
  - Les auteurs de publications sont des chercheurs référencés
  - L'ordre des auteurs est unique par publication
4. **Unicité garantie** : Emails, ORCID, DOI sont uniques grâce aux mécanismes de détection de doublons
5. **Réalisme des données** : Utilisation de Faker avec locale française pour des noms, adresses et domaines cohérents

### 3.4.1 Statistiques des données générées

Tableau récapitulatif du nombre d'enregistrements par table pour un paramètre `nb_rows` fixé à 50 000.

| Table              | Nombre d'enregistrements |
|--------------------|--------------------------|
| Institution        | 5 000                    |
| Laboratoire        | 6 250                    |
| Chercheur          | 50 000                   |
| Projet             | 10 000                   |
| Contrat            | 12 460                   |
| Publication        | 10 000                   |
| Publication_Auteur | 29 818                   |
| Jeu_Donnees        | 6 150                    |

TABLE 10 – Statistiques des données générées pour 50 000 chercheurs.



## 3.5. Exemples de code

### 3.5.1 Génération des projets

```

1 def generate_projets(f, fake: Faker, laboratoires: List[int], chercheurs
  : List[int], count: int) -> List[int]:
2     """Genere les projets et retourne la liste des IDs."""
3     projets = []
4
5     for pid in range(1, count + 1):
6         titre = fake.catch_phrase()
7         description = fake.paragraph(nb_sentences=3)
8         discipline = random.choice(["Physique", "Biologie", "
          Informatique", "Chimie", "Mathematiques"])
9         budget = round(random.uniform(10000, 500000), 2)
10        d_debut = fake.date_between(start_date='-5y', end_date='today')
11        d_fin = None if random.random() < 0.2 else fake.date_between(
          start_date=d_debut, end_date='+3y')
12        id_labo = random.choice(laboratoires)
13        id_resp = random.choice(chercheurs) if chercheurs else None
14        projets.append(pid)
15        f.write(f"INSERT INTO univ_recherche.projet (id, titre,
          description, discipline, budget_annuel_eur, date_debut,
          date_fin, id_laboratoire_pilote, id_chercheur_responsable)
          VALUES ({pid}, {_quote_sql(titre)}, {_quote_sql(description)}
          }, {_quote_sql(discipline)}, {budget}, {_quote_sql(d_debut)},
          {_quote_sql(d_fin)}, {id_labo}, {_quote_sql(id_resp)});\n")
16
17    f.write("SELECT setval(pg_get_serial_sequence('univ_recherche.projet
          ','id'), COALESCE((SELECT MAX(id) FROM univ_recherche.projet),0))
          ;\n")
18    return projets

```

Listing 1 – Exemple de génération des projets

### 3.5.2 Génération des chercheurs

```

1 def generate_chercheurs(f, fake: Faker, laboratoires: List[int], count:
  int) -> List[int]:
2     """Genere les chercheurs et retourne la liste des IDs."""
3     chercheurs = []
4     emails_used = set()
5     orcid_ids_used = set()
6
7     for cid in range(1, count + 1):
8         prenom = fake.first_name()
9         nom = fake.last_name()
10
11        # Generer un email unique
12        attempts = 0
13        while attempts < 100:
14            if attempts == 0:
15                email = (prenom + "." + nom + "@" + fake.domain_name()).
                  lower()

```

```

16         else:
17             email = (prenom + "." + nom + str(attempts) + "@" + fake
18                     .domain_name()).lower()
19             if email not in emails_used:
20                 emails_used.add(email)
21                 break
22             attempts += 1
23         else:
24             # Fallback: email avec ID unique
25             email = f"chercheur{cid}@" + fake.domain_name()
26             emails_used.add(email)
27
28             # Generer un ORCID unique ou NULL
29             orcid = None
30             if random.random() < 0.5:
31                 attempts = 0
32                 while attempts < 100:
33                     orcid_candidate = gen_orcid(fake)
34                     if orcid_candidate not in orcids_used:
35                         orcids_used.add(orcid_candidate)
36                         orcid = orcid_candidate
37                         break
38                 attempts += 1
39
40             discipline = random.choice(["Physique", "Biologie", "
41                                         Informatique", "Chimie", "Mathematiques"])
42             id_labo = random.choice(laboratoires)
43             chercheurs.append(cid)
44             f.write(f"INSERT INTO univ_recherche.chercheur (id, prenom, nom,
45                     email, orcid, discipline, id_laboratoire) VALUES ({cid}, {
46                         _quote_sql(prenom)}, {_quote_sql(nom)}, {_quote_sql(email)},
47                         {_quote_sql(orcid)}, {_quote_sql(discipline)}, {id_labo});\n"
48                     )
49
50             f.write("SELECT setval(pg_get_serial_sequence('univ_recherche.
51                     chercheur','id'), COALESCE((SELECT MAX(id) FROM univ_recherche.
52                     chercheur),0));\n")
53
54             return chercheurs

```

Listing 2 – Exemple de génération des chercheurs

## 4. Partie 3 - Gestion des Utilisateurs et Création de Vues

### 4.1. Stratégie de sécurité

#### 4.1.1 Objectifs

Pour pouvoir mettre en oeuvre une base de données réaliste, elle doit être sécurisée. Nous avons alors, en suivant rigoureusement les besoins de l'énoncé, mis en place une stratégie de sécurité basée sur la gestion des rôles et des privilèges.

#### 4.1.2 Architecture des rôles

Nous avons défini trois rôles principaux pour gérer les accès à la base de données :

- **Chercheur** : Accès restreint aux données liées à ses propres projets et publications.
- **Data Manager** : Accès élargi aux métadonnées des jeux de données et DMP, mais sans droits de suppression.
- **Administrateur** : Accès complet à toutes les données et fonctionnalités de la base.

### 4.2. Définition des rôles

#### 4.2.1 Rôle Chercheur

- Description : Accès restreint aux données du chercheur
- Privilèges : SELECT, INSERT, UPDATE sur ses propres données
- Restrictions : Pas d'accès aux données des autres chercheurs

#### 4.2.2 Rôle Data Manager

- Description : Accès élargi aux métadonnées
- Privilèges : SELECT sur toutes les tables, INSERT/UPDATE sur jeux de données et DMP
- Restrictions : Pas de suppression de données

#### 4.2.3 Rôle Administrateur

- Description : Accès complet
- Privilèges : ALL PRIVILEGES
- Responsabilités : Gestion complète de la base

### 4.3. Vues créées

#### 4.3.1 Vue 1 : Mes\_Projets

- Objectif : Permettre à un chercheur de voir ses projets
- Utilisateurs cibles : Chercheurs
- Contenu : Projets auxquels le chercheur participe

```

1 CREATE OR REPLACE VIEW v_mes_projets AS
2 SELECT
3     p.id AS projet_id,
4     p.titre,
5     p.discipline,
6     p.date_debut,
7     p.date_fin,
8     p.budget_annuel_eur,
9     l.nom AS laboratoire,
10    CONCAT(resp.prenom, ' ', resp.nom) AS responsable,
11    pc.role AS mon_role,
12    pc.charge_pct
13 FROM projet p
14 JOIN laboratoire l ON p.id_laboratoire_pilote = l.id
15 JOIN projet_chercheur pc ON p.id = pc.id_projet
16 LEFT JOIN chercheur resp ON p.id_chercheur_responsable = resp.id
17 WHERE pc.id_chercheur = (SELECT id FROM chercheur WHERE email =
    current_user);

```

Listing 3 – Vue Mes\_Projets

#### 4.3.2 Vue 2 : Metadata\_Jeux\_Donnees

- Objectif : Vue complète des métadonnées des jeux de données
- Utilisateurs cibles : Data Managers
- Contenu : Toutes les informations sur les jeux de données

#### 4.3.3 Vue 3 : Publications\_Par\_Laboratoire

- Objectif : Statistiques des publications par laboratoire
- Utilisateurs cibles : Administrateurs, Data Managers

#### 4.3.4 Vue 4 : Contrats\_Actifs

- Objectif : Liste des contrats en cours
- Utilisateurs cibles : Tous les rôles

#### 4.3.5 Vue 5 : Projets\_Budget\_Global

- Objectif : Vue agrégée des budgets par projet
- Utilisateurs cibles : Administrateurs

Voir Annexe pour le code SQL complet des vues.

### 4.4. Attribution des privilèges

#### 4.4.1 Privilèges par rôle

Tableau récapitulatif des privilèges accordés à chaque rôle.

| Vue/Table             | Chercheur | Data Manager   | Admin |
|-----------------------|-----------|----------------|-------|
| mes_projets           | SELECT    | SELECT         | ALL   |
| metadata_jeux_donnees | -         | SELECT, INSERT | ALL   |
| publications_par_labo | -         | SELECT         | ALL   |
| contrats_actifs       | SELECT    | SELECT         | ALL   |
| projets_budget_global | -         | -              | ALL   |

TABLE 11 – Attribution des privilèges par rôle

#### 4.4.2 Exemples de scripts SQL

```

1  -- Role Chercheur
2  CREATE ROLE role_chercheur;
3
4  -- Role Data Manager
5  CREATE ROLE role_data_manager;
6
7  -- Role Administrateur
8  CREATE ROLE role_administrateur;
9
10 -- Creation des vues, etc. (voir sections precedentes)
11
12 -- Privileges pour ADMINISTRATEUR
13 GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA univ_recherche TO
    role_administrateur;
14 GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA univ_recherche TO
    role_administrateur;
15 GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA univ_recherche TO
    role_administrateur;
16
17 -- Privileges pour DATA_MANAGER
18 GRANT SELECT ON ALL TABLES IN SCHEMA univ_recherche TO role_data_manager
    ;
19 GRANT INSERT, UPDATE, DELETE ON contrat, jeu_donnees TO
    role_data_manager;
20 GRANT SELECT ON v_metadonnees_contrats, v_synthese_donnees_projet,
    v_tableau_bord_projets TO role_data_manager;
21 GRANT SELECT ON v_publications_chercheur TO role_data_manager;
22
23 -- Privileges pour CHERCHEUR
24 GRANT SELECT ON projet, chercheur, laboratoire, institution, publication
    TO role_chercheur;
25 GRANT SELECT ON v_mes_projets, v_mes_donnees, v_publications_chercheur
    TO role_chercheur;
26 GRANT INSERT ON publication, publication_auteur, jeu_donnees TO
    role_chercheur;
27 GRANT UPDATE ON jeu_donnees TO role_chercheur;

```

Listing 4 – Création des rôles et attribution des privilèges

## 5. Partie 4 - Requêtes et Optimisation

### 5.1. Méthodologie d'optimisation

#### 5.1.1 Démarche

1. Écriture de plusieurs versions de chaque requête
2. Analyse des plans d'exécution
3. Comparaison des performances et des méthodes utilisées par le SGBD
4. Création d'index ciblés
5. Validation des améliorations

### 5.2. Requête R1 - Jeux de données par projet

#### 5.2.1 Énoncé

Nombre total de jeux de données déposés par projet et par année civile, ainsi que la moyenne du délai de dépôt (en jours) pour les projets ayant impliqué  $> 5$  chercheurs depuis 2018.

#### 5.2.2 Version 1 : Utilisation d'une sous-requête avec WITH

```
1  WITH projets_eligibles AS (  
2      SELECT p.id AS id_projet  
3      FROM projet p  
4      JOIN projet_chercheur pc ON p.id = pc.id_projet  
5      WHERE p.date_debut >= '2018-01-01'  
6      GROUP BY p.id  
7      HAVING COUNT(DISTINCT pc.id_chercheur) > 5  
8  )  
9  SELECT  
10     p.id AS projet_id,  
11     p.titre AS projet_titre,  
12     EXTRACT(YEAR FROM jd.date_depot) AS annee_civile,  
13     COUNT(jd.id) AS nb_jeux_donnees_deposes,  
14     ROUND(AVG(jd.date_depot - jd.date_creation), 2) AS  
15         delai_moyen_jours  
16 FROM projet p  
17 JOIN contrat ct ON p.id = ct.id_projet  
18 JOIN jeu_donnees jd ON ct.id = jd.id_contrat  
19 WHERE p.id IN (SELECT id_projet FROM projets_eligibles)  
20 AND jd.date_depot IS NOT NULL  
    GROUP BY p.id, p.titre, EXTRACT(YEAR FROM jd.date_depot)
```

Listing 5 – R1 - Version 1

#### 5.2.3 Version 2 : Jointures

```
1  SELECT
2      p.id AS projet_id,
3      p.titre AS projet_titre,
4      EXTRACT(YEAR FROM jd.date_depot) AS annee_civile,
5      COUNT(DISTINCT jd.id) AS nb_jeux_donnees_deposes,
6      ROUND(AVG(jd.date_depot - jd.date_creation), 2) AS
7          delai_moyen_jours
8  FROM projet p
9  JOIN projet_chercheur pc ON p.id = pc.id_projet
10 JOIN contrat ct ON p.id = ct.id_projet
11 JOIN jeu_donnees jd ON ct.id = jd.id_contrat
12 WHERE p.date_debut >= '2018-01-01'
13 AND jd.date_depot IS NOT NULL
14 GROUP BY p.id, p.titre, EXTRACT(YEAR FROM jd.date_depot)
15 HAVING COUNT(DISTINCT pc.id_chercheur) > 5
```

Listing 6 – R1 - Version 2

#### 5.2.4 Analyse des performances

Ces deux versions ont été testées sur un ensemble de données volumineux, et sans index d'optimisation. Les résultats des coûts d'exécution sont les suivants :

- Coûts d'exécution Version 1 : 3792.40
- Utilisation de scans séquentiels principalement.
- Coûts d'exécution Version 2 : 3765.40
- Utilisation de scans séquentiels principalement.

La seconde version est légèrement plus performante, probablement en raison de l'optimisation des jointures par le SGBD.

### 5.2.5 Plans d'exécution

|                                                                                                                                                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sort (cost=3789.61..3792.40 rows=1115 width=119) (actual time=29.076..29.097 rows=381 loops=1)                                                             |
| Sort Key: p.titre, (EXTRACT(year FROM jd.date_depot)) DESC                                                                                                 |
| Sort Method: quicksort Memory: 58kB                                                                                                                        |
| -> HashAggregate (cost=3713.66..3733.17 rows=1115 width=119) (actual time=28.389..28.511 rows=381 loops=1)                                                 |
| Group Key: EXTRACT(year FROM jd.date_depot), p.id                                                                                                          |
| Batches: 1 Memory Usage: 193kB                                                                                                                             |
| -> Hash Join (cost=3253.72..3699.72 rows=1115 width=95) (actual time=26.898..28.213 rows=553 loops=1)                                                      |
| Hash Cond: (p.id = ct.id_projet)                                                                                                                           |
| -> Seq Scan on projet p (cost=0.00..390.00 rows=10000 width=47) (actual time=0.012..0.534 rows=10000 loops=1)                                              |
| -> Hash (cost=3234.07..3234.07 rows=1572 width=32) (actual time=26.860..26.865 rows=553 loops=1)                                                           |
| Buckets: 2048 Batches: 1 Memory Usage: 55kB                                                                                                                |
| -> Hash Join (cost=2770.39..3234.07 rows=1572 width=32) (actual time=23.135..26.760 rows=553 loops=1)                                                      |
| Hash Cond: (ct.id = jd.id_contrat)                                                                                                                         |
| -> Hash Join (cost=2459.26..2841.10 rows=5877 width=24) (actual time=19.005..22.258 rows=2058 loops=1)                                                     |
| Hash Cond: (ct.id_projet = p_1.id)                                                                                                                         |
| -> Seq Scan on contrat ct (cost=0.00..349.00 rows=12500 width=16) (actual time=0.019..1.256 rows=12500 loops=1)                                            |
| -> Hash (cost=2417.60..2417.60 rows=3333 width=8) (actual time=18.930..18.932 rows=1694 loops=1)                                                           |
| Buckets: 4096 Batches: 1 Memory Usage: 99kB                                                                                                                |
| -> GroupAggregate (cost=0.57..2417.60 rows=3333 width=8) (actual time=0.073..18.562 rows=1694 loops=1)                                                     |
| Group Key: p_1.id                                                                                                                                          |
| Filter: (count(DISTINCT pc.id_chercheur) > 5)                                                                                                              |
| Rows Removed by Filter: 8306                                                                                                                               |
| -> Merge Join (cost=0.57..2118.01 rows=34921 width=16) (actual time=0.050..14.107 rows=34924 loops=1)                                                      |
| Merge Cond: (pc.id_projet = p_1.id)                                                                                                                        |
| -> Index Only Scan using uq_projet_chercheur on projet_chercheur pc (cost=0.29..1068.15 rows=34924 width=16) (actual time=0.030..4.743 rows=34924 loops=1) |
| Heap Fetches: 0                                                                                                                                            |
| -> Index Scan using projet_pkey on projet p_1 (cost=0.29..588.34 rows=9999 width=8) (actual time=0.016..4.314 rows=10000 loops=1)                          |
| Filter: (date_debut >= '2018-01-01'::date)                                                                                                                 |
| -> Hash (cost=269.33..269.33 rows=3344 width=24) (actual time=4.096..4.097 rows=3344 loops=1)                                                              |
| Buckets: 4096 Batches: 1 Memory Usage: 241kB                                                                                                               |
| -> Seq Scan on jeu_donnees jd (cost=0.00..269.33 rows=3344 width=24) (actual time=0.012..3.394 rows=3344 loops=1)                                          |
| Filter: (date_depot IS NOT NULL)                                                                                                                           |
| Rows Removed by Filter: 4989                                                                                                                               |

FIGURE 2 – Plan d'exécution de la Version 1 de la requête R1 sans index d'optimisation.



|                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------|
| Sort (cost=3755.67..3765.40 rows=3893 width=119) (actual time=61.969..61.993 rows=381 loops=1)                           |
| Sort Key: p.titre, (EXTRACT(year FROM jd.date_depot)) DESC                                                               |
| Sort Method: quicksort Memory: 58kB                                                                                      |
| -> GroupAggregate (cost=3124.52..3523.52 rows=3893 width=119) (actual time=55.722..61.274 rows=381 loops=1)              |
| Group Key: (EXTRACT(year FROM jd.date_depot)), p.id                                                                      |
| Filter: (count(DISTINCT pc.id_chercheur) > 5)                                                                            |
| Rows Removed by Filter: 1993                                                                                             |
| -> Sort (cost=3124.52..3153.71 rows=11678 width=103) (actual time=55.531..56.107 rows=11769 loops=1)                     |
| Sort Key: (EXTRACT(year FROM jd.date_depot)) DESC, p.id, jd.id                                                           |
| Sort Method: quicksort Memory: 1495kB                                                                                    |
| -> Hash Join (cost=1365.39..2335.58 rows=11678 width=103) (actual time=16.871..37.478 rows=11769 loops=1)                |
| Hash Cond: (pc.id_projet = p.id)                                                                                         |
| -> Seq Scan on projet_chercheur pc (cost=0.00..693.24 rows=34924 width=16) (actual time=0.021..7.295 rows=34924 loops=1) |
| -> Hash (cost=1323.59..1323.59 rows=3344 width=71) (actual time=16.661..16.665 rows=3344 loops=1)                        |
| Buckets: 4096 Batches: 1 Memory Usage: 398kB                                                                             |
| -> Hash Join (cost=825.16..1323.59 rows=3344 width=71) (actual time=10.336..15.000 rows=3344 loops=1)                    |
| Hash Cond: (p.id = ct.id_projet)                                                                                         |
| -> Seq Scan on projet p (cost=0.00..415.00 rows=9999 width=47) (actual time=0.009..3.003 rows=10000 loops=1)             |
| Filter: (date_debut >= '2018-01-01'::date)                                                                               |
| -> Hash (cost=783.36..783.36 rows=3344 width=24) (actual time=10.268..10.270 rows=3344 loops=1)                          |
| Buckets: 4096 Batches: 1 Memory Usage: 228kB                                                                             |
| -> Hash Join (cost=505.25..783.36 rows=3344 width=24) (actual time=6.723..9.340 rows=3344 loops=1)                       |
| Hash Cond: (jd.id_contrat = ct.id)                                                                                       |
| -> Seq Scan on jeu_donnees jd (cost=0.00..269.33 rows=3344 width=24) (actual time=0.007..1.850 rows=3344 loops=1)        |
| Filter: (date_depot IS NOT NULL)                                                                                         |
| Rows Removed by Filter: 4989                                                                                             |
| -> Hash (cost=349.00..349.00 rows=12500 width=16) (actual time=6.669..6.670 rows=12500 loops=1)                          |
| Buckets: 16384 Batches: 1 Memory Usage: 714kB                                                                            |
| -> Seq Scan on contrat ct (cost=0.00..349.00 rows=12500 width=16) (actual time=0.037..4.352 rows=12500 loops=1)          |

FIGURE 3 – Plan d'exécution de la Version 2 de la requête R1 sans index d'optimisation.

Les deux versions utilisent des scans séquentiels sur les tables principales, expliquant probablement les temps d'exécution élevés.

En y ajoutant des index ciblés :

```

1 CREATE INDEX IF NOT EXISTS idx_jeu_donnees_date_depot ON jeu_donnees
   (date_depot);
2 CREATE INDEX IF NOT EXISTS idx_jeu_donnees_date_creation ON
   jeu_donnees(date_creation);
3 CREATE INDEX IF NOT EXISTS idx_jeu_donnees_contrat ON jeu_donnees(
   id_contrat);
4 CREATE INDEX IF NOT EXISTS idx_contrat_projet ON contrat(id_projet);
5
6 CREATE INDEX IF NOT EXISTS idx_projet_chercheur_projet ON
   projet_chercheur(id_projet);
7 CREATE INDEX IF NOT EXISTS idx_projet_chercheur_chercheur ON
   projet_chercheur(id_chercheur);

```

Listing 7 – Mise en place des index pour R1

Les deux versions ne semblent pas voir de différence significative dans les plans d'exécution. Bien que le temps d'exécution ait été réduit (de 10ms environ chacun), les coûts, eux, restent assez proches :

- Coûts d'exécution Version 1 avec index : 3788.77 (comparé à 3792.40 sans index)
- Le plan d'exécution reste similaire, avec des scans séquentiels toujours présents.

- Coûts d'exécution Version 2 avec index : 3761.83 (comparé à 3765.40 sans index)
- Le plan d'exécution reste similaire, avec des scans séquentiels toujours présents.

### 5.2.6 Optimisation choisie

Après analyse, nous avons choisi la Version 2 de la requête R1 pour son coût légèrement inférieur. Nous avons également mis en place les index listés ci-dessous pour améliorer les performances globales des requêtes similaires.

## 5.3. Requête R2 - Projets sans chercheurs peu productifs

### 5.3.1 Énoncé

Pour le laboratoire "Unleash clicks-and-mortar solutions Lab" en 2024, donner la liste des projets et le nom du responsable qui n'ont aucun chercheur dont le nombre de publications en 2024 est strictement inférieur à la moyenne.

### 5.3.2 Version 1 : Utilisation de NOT IN

```

1  SELECT
2      p.id AS projet_id,
3      p.titre AS projet_titre,
4      CONCAT(resp.prenom, ' ', resp.nom) AS responsable
5  FROM projet p
6  JOIN laboratoire l ON p.id_laboratoire_pilote = l.id
7  LEFT JOIN chercheur resp ON p.id_chercheur_responsable = resp.id
8  WHERE l.nom = 'Unleash clicks-and-mortar solutions Lab'
9  AND (p.date_debut <= '2024-12-31'
10     AND (p.date_fin IS NULL OR p.date_fin >= '2024-01-01'))
11  AND p.id NOT IN (
12      SELECT DISTINCT pc.id_projet
13      FROM projet_chercheur pc
14      JOIN chercheur c ON pc.id_chercheur = c.id
15      JOIN laboratoire l2 ON c.id_laboratoire = l2.id
16      LEFT JOIN publication_auteur pa ON c.id = pa.id_chercheur
17      LEFT JOIN publication pub ON pa.id_publication = pub.id
18           AND EXTRACT(YEAR FROM pub.date_publication) = 2024
19      WHERE l2.nom = 'Unleash clicks-and-mortar solutions Lab'
20      GROUP BY pc.id_projet, c.id
21      HAVING COUNT(DISTINCT pub.id) < (
22          SELECT AVG(cnt)
23          FROM (
24              SELECT COUNT(DISTINCT pub2.id) AS cnt
25              FROM chercheur c2
26              JOIN laboratoire l3 ON c2.id_laboratoire = l3.id
27              LEFT JOIN publication_auteur pa2 ON c2.id = pa2.
28                  id_chercheur
29              LEFT JOIN publication pub2 ON pa2.id_publication = pub2.
30                  id
31              AND EXTRACT(YEAR FROM pub2.date_publication) = 2024
32              WHERE l3.nom = 'Unleash clicks-and-mortar solutions Lab'
33              GROUP BY c2.id
34          ) AS moyennes

```

```

33     )
34 )

```

Listing 8 – R2 - Version 1

### 5.3.3 Version 2 : Utilisation de EXCEPT

```

1  SELECT
2      p.id AS projet_id,
3      p.titre AS projet_titre,
4      CONCAT(resp.prenom, ' ', resp.nom) AS responsable
5  FROM projet p
6  JOIN laboratoire l ON p.id_laboratoire_pilote = l.id
7  LEFT JOIN chercheur resp ON p.id_chercheur_responsable = resp.id
8  WHERE l.nom = 'Unleash clicks-and-mortar solutions Lab'
9  AND (p.date_debut <= '2024-12-31'
10      AND (p.date_fin IS NULL OR p.date_fin >= '2024-01-01'))
11
12  EXCEPT
13
14  SELECT DISTINCT
15      p.id AS projet_id,
16      p.titre AS projet_titre,
17      CONCAT(resp.prenom, ' ', resp.nom) AS responsable
18  FROM projet p
19  JOIN laboratoire l ON p.id_laboratoire_pilote = l.id
20  LEFT JOIN chercheur resp ON p.id_chercheur_responsable = resp.id
21  JOIN projet_chercheur pc ON p.id = pc.id_projet
22  JOIN chercheur c ON pc.id_chercheur = c.id
23  LEFT JOIN publication_auteur pa ON c.id = pa.id_chercheur
24  LEFT JOIN publication pub ON pa.id_publication = pub.id
25      AND EXTRACT(YEAR FROM pub.date_publication) = 2024
26  WHERE l.nom = 'Unleash clicks-and-mortar solutions Lab'
27  AND c.id_laboratoire = l.id
28  GROUP BY p.id, p.titre, resp.prenom, resp.nom, c.id
29  HAVING COUNT(DISTINCT pub.id) < (
30      SELECT AVG(cnt)
31      FROM (
32          SELECT COUNT(DISTINCT pub2.id) AS cnt
33          FROM chercheur c2
34          JOIN laboratoire l2 ON c2.id_laboratoire = l2.id
35          LEFT JOIN publication_auteur pa2 ON c2.id = pa2.id_chercheur
36          LEFT JOIN publication pub2 ON pa2.id_publication = pub2.id
37              AND EXTRACT(YEAR FROM pub2.date_publication) = 2024
38          WHERE l2.nom = 'Unleash clicks-and-mortar solutions Lab'
39          GROUP BY c2.id
40      ) AS stats_labo
41  )

```

Listing 9 – R2 - Version 2

### 5.3.4 Analyse des performances

Les deux versions ont été testées sur un ensemble de données volumineux, et sans index d'optimisation. Les résultats des coûts d'exécution sont les suivants :

- Coûts d'exécution Version 1 : 5146.99
- Utilisation de scans séquentiels et hash joins principalement.
- Coûts d'exécution Version 2 : 3039.05
- Utilisation de scans séquentiels et hash joins.

La version avec EXCEPT est plus performante, probablement en raison d'une meilleure optimisation des ensembles par le SGBD.

### 5.3.5 Plans d'exécution

|                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nested Loop Left Join (cost=4672.71..5146.99 rows=1 width=79) (actual time=42.389..44.540 rows=2 loops=1)                                                |
| -> Hash Join (cost=4672.42..5145.64 rows=1 width=55) (actual time=42.362..44.506 rows=2 loops=1)                                                         |
| Hash Cond: (p.id_laboratoire_pilote = l.id)                                                                                                              |
| -> Seq Scan on projet p (cost=4664.11..5129.11 rows=3126 width=63) (actual time=42.286..44.018 rows=7227 loops=1)                                        |
| Filter: ((date_debut <= '2024-12-31'::date) AND ((date_fin IS NULL) OR (date_fin >= '2024-01-01'::date)) AND (NOT (ANY (id = (hashed SubPlan 2).col1)))) |
| Rows Removed by Filter: 2773                                                                                                                             |
| SubPlan 2                                                                                                                                                |
| -> Unique (cost=4663.95..4664.10 rows=2 width=16) (actual time=42.217..42.241 rows=6 loops=1)                                                            |
| InitPlan 1                                                                                                                                               |
| -> Aggregate (cost=1920.26..1920.27 rows=1 width=32) (actual time=10.856..10.865 rows=1 loops=1)                                                         |
| -> GroupAggregate (cost=1920.02..1920.16 rows=8 width=16) (actual time=10.844..10.857 rows=12 loops=1)                                                   |
| Group Key: c2.id                                                                                                                                         |
| -> Sort (cost=1920.02..1920.04 rows=8 width=16) (actual time=10.837..10.845 rows=17 loops=1)                                                             |
| Sort Key: c2.id, pub2.id                                                                                                                                 |
| Sort Method: quicksort Memory: 25kB                                                                                                                      |
| -> Nested Loop Left Join (cost=1318.04..1919.90 rows=8 width=16) (actual time=7.767..10.828 rows=17 loops=1)                                             |
| -> Hash Right Join (cost=1317.75..1917.26 rows=8 width=16) (actual time=7.742..10.699 rows=17 loops=1)                                                   |
| Hash Cond: (pa2.id_chercheur = c2.id)                                                                                                                    |
| -> Seq Scan on publication_auteur pa2 (cost=0.00..487.79 rows=29779 width=16) (actual time=0.014..1.407 rows=29779 loops=1)                              |
| -> Hash (cost=1317.65..1317.65 rows=8 width=8) (actual time=7.656..7.661 rows=12 loops=1)                                                                |
| Buckets: 1024 Batches: 1 Memory Usage: 9kB                                                                                                               |
| -> Hash Join (cost=8.31..1317.65 rows=8 width=8) (actual time=0.908..7.632 rows=12 loops=1)                                                              |
| Hash Cond: (c2.id_laboratoire = l3.id)                                                                                                                   |
| -> Seq Scan on chercheur c2 (cost=0.00..1178.00 rows=50000 width=16) (actual time=0.015..2.887 rows=50000 loops=1)                                       |
| -> Hash (cost=8.30..8.30 rows=1 width=8) (actual time=0.020..0.021 rows=1 loops=1)                                                                       |
| Buckets: 1024 Batches: 1 Memory Usage: 9kB                                                                                                               |
| -> Index Scan using uq_laboratoire_nom_institution on laboratoire l3 (cost=0.28..8.30 rows=1 width=8) (actual time=0.000..0.000 rows=1 loops=1)          |
| Index Cond: (nom = 'Unleash clicks-and-mortar solutions Lab'::text)                                                                                      |
| -> Index Scan using publication_pkey on publication pub2 (cost=0.29..0.33 rows=1 width=8) (actual time=0.007..0.007 rows=0 loops=1)                      |

FIGURE 4 – Plan d'exécution de la Version 1 de la requête R2 sans index d'optimisation.

|                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------|
| HashSetOp Except (cost=8.60..3039.05 rows=1 width=76) (actual time=3.960..3.971 rows=2 loops=1)                                                |
| -> Append (cost=8.60..3039.03 rows=2 width=76) (actual time=0.146..3.967 rows=2 loops=1)                                                       |
| -> Subquery Scan on "SELECT* 1" (cost=8.60..465.58 rows=1 width=83) (actual time=0.146..2.222 rows=2 loops=1)                                  |
| -> Nested Loop Left Join (cost=8.60..465.57 rows=1 width=79) (actual time=0.145..2.219 rows=2 loops=1)                                         |
| -> Hash Join (cost=8.31..464.74 rows=1 width=55) (actual time=0.104..2.160 rows=2 loops=1)                                                     |
| Hash Cond: (p.id_laboratoire_pilote = l.id)                                                                                                    |
| -> Seq Scan on projet p (cost=0.00..440.00 rows=6253 width=63) (actual time=0.016..1.640 rows=7232 loops=1)                                    |
| Filter: ((date_debut <= '2024-12-31'::date) AND ((date_fin IS NULL) OR (date_fin >= '2024-01-01'::date)))                                      |
| Rows Removed by Filter: 2768                                                                                                                   |
| -> Hash (cost=8.30..8.30 rows=1 width=8) (actual time=0.072..0.073 rows=1 loops=1)                                                             |
| Buckets: 1024 Batches: 1 Memory Usage: 9kB                                                                                                     |
| -> Index Scan using uq_laboratoire_nom_institution on laboratoire l (cost=0.28..8.30 rows=1 width=8) (actual time=0.069..0.070 rows=1 loops=1) |
| Index Cond: (nom = 'Unleash clicks-and-mortar solutions Lab'::text)                                                                            |
| -> Index Scan using chercheur_pkey on chercheur resp (cost=0.29..0.83 rows=1 width=23) (actual time=0.025..0.025 rows=1 loops=2)               |
| Index Cond: (id = p.id_chercheur_responsable)                                                                                                  |
| -> Subquery Scan on "SELECT* 2" (cost=2573.32..2573.44 rows=1 width=83) (actual time=1.736..1.742 rows=0 loops=1)                              |
| -> Unique (cost=2573.32..2573.43 rows=1 width=102) (actual time=1.735..1.741 rows=0 loops=1)                                                   |
| InitPlan 1                                                                                                                                     |
| -> Aggregate (cost=1920.26..1920.27 rows=1 width=32) (never executed)                                                                          |
| -> GroupAggregate (cost=1920.02..1920.16 rows=8 width=16) (never executed)                                                                     |
| Group Key: c2.id                                                                                                                               |
| -> Sort (cost=1920.02..1920.04 rows=8 width=16) (never executed)                                                                               |
| Sort Key: c2.id, pub2.id                                                                                                                       |
| -> Nested Loop Left Join (cost=1318.04..1919.90 rows=8 width=16) (never executed)                                                              |
| -> Hash Right Join (cost=1317.75..1917.26 rows=8 width=16) (never executed)                                                                    |
| Hash Cond: (pa2.id_chercheur = c2.id)                                                                                                          |
| -> Seq Scan on publication_auteur pa2 (cost=0.00..487.79 rows=29779 width=16) (never executed)                                                 |
| -> Hash (cost=1317.65..1317.65 rows=8 width=8) (never executed)                                                                                |

FIGURE 5 – Plan d'exécution de la Version 2 de la requête R2 sans index d'optimisation.

### 5.3.6 Optimisation choisie

Après analyse, nous avons choisi la Version 2 de la requête R2 pour son coût nettement inférieur. Nous avons également mis en place les index listés ci-dessous pour améliorer les performances globales des requêtes similaires.

```

1 CREATE INDEX IF NOT EXISTS idx_publication_date_publication ON
   publication(date_publication);
2 CREATE INDEX IF NOT EXISTS idx_publication_auteur_chercheur ON
   publication_auteur(id_chercheur);
3 CREATE INDEX IF NOT EXISTS idx_publication_auteur_publication ON
   publication_auteur(id_publication);
4 CREATE INDEX IF NOT EXISTS idx_chercheur_laboratoire ON chercheur(
   id_laboratoire);

```

Listing 10 – Mise en place des index pour R2

Nous avons constaté une très grande amélioration des performances après l'ajout de ces index, avec une réduction significative du coût d'exécution (de 3039.05 à 943.38).

|                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------|
| HashSetOp Except (cost=8.60..943.38 rows=1 width=76) (actual time=7.357..7.385 rows=2 loops=1)                                                 |
| -> Append (cost=8.60..943.36 rows=2 width=76) (actual time=0.103..7.358 rows=2 loops=1)                                                        |
| -> Subquery Scan on "SELECT* 1" (cost=8.60..465.58 rows=1 width=83) (actual time=0.103..3.864 rows=2 loops=1)                                  |
| -> Nested Loop Left Join (cost=8.60..465.57 rows=1 width=79) (actual time=0.101..3.860 rows=2 loops=1)                                         |
| -> Hash Join (cost=8.31..464.74 rows=1 width=55) (actual time=0.082..3.819 rows=2 loops=1)                                                     |
| Hash Cond: (p.id_laboratoire_pilote = l.id)                                                                                                    |
| -> Seq Scan on projet p (cost=0.00..440.00 rows=6253 width=63) (actual time=0.027..2.829 rows=7232 loops=1)                                    |
| Filter: ((date_debut <= '2024-12-31'::date) AND ((date_fin IS NULL) OR (date_fin >= '2024-01-01'::date)))                                      |
| Rows Removed by Filter: 2768                                                                                                                   |
| -> Hash (cost=8.30..8.30 rows=1 width=8) (actual time=0.045..0.050 rows=1 loops=1)                                                             |
| Buckets: 1024 Batches: 1 Memory Usage: 9kB                                                                                                     |
| -> Index Scan using uq_laboratoire_nom_institution on laboratoire l (cost=0.28..8.30 rows=1 width=8) (actual time=0.039..0.041 rows=1 loops=1) |
| Index Cond: (nom = 'Unleash clicks-and-mortar solutions Lab'::text)                                                                            |
| -> Index Scan using chercheur_pkey on chercheur resp (cost=0.29..0.83 rows=1 width=23) (actual time=0.010..0.011 rows=1 loops=2)               |
| Index Cond: (id = p.id_chercheur_responsable)                                                                                                  |
| -> Subquery Scan on "SELECT* 2" (cost=477.65..477.77 rows=1 width=83) (actual time=3.471..3.483 rows=0 loops=1)                                |
| Index Cond: (id = p.id_chercheur_responsable)                                                                                                  |

FIGURE 6 – Plan d'exécution de la Version 2 de la requête R2 avec index d'optimisation.

On observe l'utilisation des Index Scan, ainsi que des Bitmap Index Scan plus profond dans le plan d'exécution, ce qui explique la réduction du coût.

## 5.4. Requête R3 - Laboratoires conformes

### 5.4.1 Énoncé

Laboratoires n'ayant eu aucun jeu de données non conforme en 2024 (sans licence ou sans date\_depot).

### 5.4.2 Version 1 : Utilisation de NOT EXISTS

```

1  SELECT
2      l.id AS laboratoire_id,
3      l.nom AS laboratoire_nom,
4      i.nom AS institution_nom
5  FROM laboratoire l
6  JOIN institution i ON l.id_institution = i.id
7  WHERE NOT EXISTS (
8      SELECT 1
9      FROM projet p
10     JOIN contrat ct ON ct.id_projet = p.id
11     JOIN jeu_donnees jd ON ct.id = jd.id_contrat
12     WHERE p.id_laboratoire_pilote = l.id
13           AND EXTRACT(YEAR FROM COALESCE(jd.date_depot, jd.date_creation))
14                 = 2024
15           AND (jd.licence IS NULL OR jd.date_depot IS NULL)
16  )

```

Listing 11 – R3 - Version 1

### 5.4.3 Version 2 : Utilisation de LEFT JOIN

```

1  SELECT
2      l.id AS laboratoire_id,
3      l.nom AS laboratoire_nom,

```

```

4      i.nom AS institution_nom
5  FROM laboratoire l
6  JOIN institution i ON l.id_institution = i.id
7  LEFT JOIN projet p ON l.id = p.id_laboratoire_pilote
8  LEFT JOIN contrat ct ON p.id = ct.id_projet
9  LEFT JOIN jeu_donnees jd ON ct.id = jd.id_contrat
10     AND EXTRACT(YEAR FROM COALESCE(jd.date_depot, jd.date_creation))
      = 2024
11     AND (jd.licence IS NULL OR jd.date_depot IS NULL)
12 GROUP BY l.id, l.nom, i.nom
13 HAVING COUNT(jd.id) = 0

```

Listing 12 – R3 - Version 2

#### 5.4.4 Analyse des performances

Les deux versions ont été testées sur un ensemble de données volumineux, et sans aucun index d'optimisation (ni même les anciens index présentés pour les deux requêtes précédentes). Les résultats des coûts d'exécution sont les suivants :

- Coûts d'exécution Version 1 : 922.05
- Utilisation de scans séquentiels et nested loop joins (traduction pour boucles imbriquées) principalement. Quelques index scans pour les jointures.
- Coûts d'exécution Version 2 : 2044.89
- Utilisation de scans séquentiels et hash joins principalement.

#### 5.4.5 Plans d'exécution

|                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------|
| Hash Join (cost=378.69..922.05 rows=6221 width=54) (actual time=15.262..16.635 rows=4808 loops=1)                                |
| Hash Cond: (l.id_institution = i.id)                                                                                             |
| -> Hash Right Anti Join (cost=205.19..732.21 rows=6221 width=50) (actual time=13.774..14.367 rows=4808 loops=1)                  |
| Hash Cond: (p.id_laboratoire_pilote = l.id)                                                                                      |
| -> Nested Loop (cost=0.57..527.51 rows=29 width=8) (actual time=0.047..11.821 rows=1956 loops=1)                                 |
| -> Nested Loop (cost=0.29..515.77 rows=29 width=8) (actual time=0.040..7.213 rows=1956 loops=1)                                  |
| -> Seq Scan on jeu_donnees jd (cost=0.00..311.00 rows=29 width=8) (actual time=0.021..2.616 rows=1956 loops=1)                   |
| Filter: (((licence IS NULL) OR (date_depot IS NULL)) AND (EXTRACT(year FROM COALESCE(date_depot, date_creation)) = '2024'::nu... |
| Rows Removed by Filter: 6377                                                                                                     |
| -> Index Scan using contrat_pkey on contrat ct (cost=0.29..7.06 rows=1 width=16) (actual time=0.002..0.002 rows=1 loops=1956)    |
| Index Cond: (id = jd.id_contrat)                                                                                                 |
| -> Index Scan using projet_pkey on projet p (cost=0.29..0.40 rows=1 width=16) (actual time=0.002..0.002 rows=1 loops=1956)       |
| Index Cond: (id = ct.id_projet)                                                                                                  |
| -> Hash (cost=126.50..126.50 rows=6250 width=50) (actual time=1.251..1.252 rows=6250 loops=1)                                    |
| Buckets: 8192 Batches: 1 Memory Usage: 591kB                                                                                     |
| -> Seq Scan on laboratoire l (cost=0.00..126.50 rows=6250 width=50) (actual time=0.006..0.485 rows=6250 loops=1)                 |
| -> Hash (cost=111.00..111.00 rows=5000 width=20) (actual time=1.469..1.469 rows=5000 loops=1)                                    |
| Buckets: 8192 Batches: 1 Memory Usage: 334kB                                                                                     |
| -> Seq Scan on institution i (cost=0.00..111.00 rows=5000 width=20) (actual time=0.009..0.563 rows=5000 loops=1)                 |
| Planning Time: 0.564 ms                                                                                                          |
| Execution Time: 16.824 ms                                                                                                        |

FIGURE 7 – Plan d'exécution de la Version 1 de la requête R3 sans index d'optimisation.

|                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------|
| Sort (cost=2044.74..2044.89 rows=62 width=54) (actual time=38.841..39.034 rows=4808 loops=1)                                       |
| Sort Key: l.nom                                                                                                                    |
| Sort Method: quicksort Memory: 543kB                                                                                               |
| -> HashAggregate (cost=1886.64..2042.89 rows=62 width=54) (actual time=27.054..28.429 rows=4808 loops=1)                           |
| Group Key: l.id, l.nom                                                                                                             |
| Filter: (count(jd.id) = 0)                                                                                                         |
| Batches: 1 Memory Usage: 1425kB                                                                                                    |
| Rows Removed by Filter: 1442                                                                                                       |
| -> Hash Left Join (cost=1204.48..1792.89 rows=12500 width=62) (actual time=8.168..21.771 rows=16811 loops=1)                       |
| Hash Cond: (ct.id = jd.id_contrat)                                                                                                 |
| -> Hash Join (cost=893.12..1340.62 rows=12500 width=62) (actual time=6.252..17.521 rows=16657 loops=1)                             |
| Hash Cond: (l.id_institution = l.id)                                                                                               |
| -> Hash Right Join (cost=719.62..1134.28 rows=12500 width=58) (actual time=4.627..12.979 rows=16657 loops=1)                       |
| Hash Cond: (p.id_laboratoire_pilote = l.id)                                                                                        |
| -> Hash Right Join (cost=515.00..896.82 rows=12500 width=16) (actual time=3.177..7.930 rows=15411 loops=1)                         |
| Hash Cond: (ct.id_projet = p.id)                                                                                                   |
| -> Seq Scan on contrat ct (cost=0.00..349.00 rows=12500 width=16) (actual time=0.009..0.871 rows=12500 loops=1)                    |
| -> Hash (cost=390.00..390.00 rows=10000 width=16) (actual time=3.080..3.083 rows=10000 loops=1)                                    |
| Buckets: 16384 Batches: 1 Memory Usage: 597kB                                                                                      |
| -> Seq Scan on projet p (cost=0.00..390.00 rows=10000 width=16) (actual time=0.011..1.701 rows=10000 loops=1)                      |
| -> Hash (cost=126.50..126.50 rows=6250 width=50) (actual time=1.409..1.409 rows=6250 loops=1)                                      |
| Buckets: 8192 Batches: 1 Memory Usage: 591kB                                                                                       |
| -> Seq Scan on laboratoire l (cost=0.00..126.50 rows=6250 width=50) (actual time=0.008..0.507 rows=6250 loops=1)                   |
| -> Hash (cost=111.00..111.00 rows=5000 width=20) (actual time=1.586..1.586 rows=5000 loops=1)                                      |
| Buckets: 8192 Batches: 1 Memory Usage: 334kB                                                                                       |
| -> Seq Scan on institution i (cost=0.00..111.00 rows=5000 width=20) (actual time=0.021..0.648 rows=5000 loops=1)                   |
| -> Hash (cost=311.00..311.00 rows=29 width=16) (actual time=1.907..1.908 rows=1956 loops=1)                                        |
| Buckets: 2048 (originally 1024) Batches: 1 (originally 1) Memory Usage: 108kB                                                      |
| -> Seq Scan on jeu_donnees jd (cost=0.00..311.00 rows=29 width=16) (actual time=0.016..1.627 rows=1956 loops=1)                    |
| Filter: (((licence IS NULL) OR (date_depot IS NULL)) AND (EXTRACT(year FROM COALESCE(date_depot, date_creation)) = '2024'::nu...)) |
| Rows Removed by Filter: 6377                                                                                                       |
| Planning Time: 0.989 ms                                                                                                            |
| Execution Time: 39.388 ms                                                                                                          |

FIGURE 8 – Plan d'exécution de la Version 2 de la requête R3 sans index d'optimisation.

#### 5.4.6 Optimisation choisie

Pour les deux versions, nous avons mis en place les index listés ci-dessous.

```

1  CREATE INDEX IF NOT EXISTS idx_jeu_donnees_date_depot ON jeu_donnees
   (date_depot);
2  CREATE INDEX IF NOT EXISTS idx_jeu_donnees_date_creation ON
   jeu_donnees(date_creation);
3  CREATE INDEX IF NOT EXISTS idx_jeu_donnees_licence ON jeu_donnees(
   licence);
4  CREATE INDEX IF NOT EXISTS idx_jeu_donnees_contrat ON jeu_donnees(
   id_contrat);
5  CREATE INDEX IF NOT EXISTS idx_contrat_projet ON contrat(id_projet);
6  CREATE INDEX IF NOT EXISTS idx_projet_laboratoire ON projet(
   id_laboratoire_pilote);

```

Listing 13 – Mise en place des index pour R3

Cependant, les deux versions ne semblent pas affectées par ces changements (2044.89 toujours pour la Version 2, et 922.05 pour la Version 1).

Après discussion avec l'IA générative, celui-ci nous a proposé un index bien plus spécifiques pour cette requête :



```

1 CREATE INDEX IF NOT EXISTS idx_jeu_donnees_non_conforme_2024
2 ON jeu_donnees(id_contrat)
3 WHERE (licence IS NULL OR date_depot IS NULL)
4 AND EXTRACT(YEAR FROM COALESCE(date_depot, date_creation)) =
    2024;

```

Listing 14 – Index spécifique pour R3

Avec cet index, la Version 1 possède désormais un coût de 615.77 au lieu de 922.05. La Version 2 possède un coût de 1815.75 au lieu de 2044.89.

Une baisse plus grande sur la Version 1, qui reste donc notre choix final pour cette requête R3.

|                                                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hash Join (cost=378.97..615.77 rows=6221 width=54) (actual time=10.538..11.823 rows=4808 loops=1)                                                           |
| Hash Cond: (l.id_institution = i.id)                                                                                                                        |
| -> Hash Right Anti Join (cost=205.47..425.93 rows=6221 width=50) (actual time=8.817..9.381 rows=4808 loops=1)                                               |
| Hash Cond: (p.id_laboratoire_pilote = l.id)                                                                                                                 |
| -> Nested Loop (cost=0.85..221.23 rows=29 width=8) (actual time=0.110..6.933 rows=1956 loops=1)                                                             |
| -> Nested Loop (cost=0.56..209.49 rows=29 width=8) (actual time=0.101..3.477 rows=1956 loops=1)                                                             |
| -> Index Only Scan using idx_jeu_donnees_non_conforme_2024 on jeu_donnees jd (cost=0.28..4.71 rows=29 width=8) (actual time=0.085..0.347 rows=1956 loops=1) |
| Heap Fetches: 0                                                                                                                                             |
| -> Index Scan using contrat_pkey on contrat ct (cost=0.29..7.06 rows=1 width=16) (actual time=0.001..0.001 rows=1 loops=1956)                               |
| Index Cond: (id = jd.id_contrat)                                                                                                                            |
| -> Index Scan using projet_pkey on projet p (cost=0.29..0.40 rows=1 width=16) (actual time=0.001..0.001 rows=1 loops=1956)                                  |
| Index Cond: (id = ct.id_projet)                                                                                                                             |
| -> Hash (cost=126.50..126.50 rows=6250 width=50) (actual time=1.345..1.346 rows=6250 loops=1)                                                               |
| Buckets: 8192 Batches: 1 Memory Usage: 591kB                                                                                                                |
| -> Seq Scan on laboratoire l (cost=0.00..126.50 rows=6250 width=50) (actual time=0.009..0.534 rows=6250 loops=1)                                            |
| -> Hash (cost=111.00..111.00 rows=5000 width=20) (actual time=1.709..1.709 rows=5000 loops=1)                                                               |
| Buckets: 8192 Batches: 1 Memory Usage: 334kB                                                                                                                |
| -> Seq Scan on institution i (cost=0.00..111.00 rows=5000 width=20) (actual time=0.008..0.807 rows=5000 loops=1)                                            |
| Planning Time: 0.777 ms                                                                                                                                     |
| Execution Time: 11.997 ms                                                                                                                                   |

FIGURE 9 – Plan d'exécution de la Version 1 de la requête R3 avec index d'optimisation.

|                                                                                                                                          |
|------------------------------------------------------------------------------------------------------------------------------------------|
| Sort (cost=1815.75..1815.90 rows=62 width=54) (actual time=36.942..37.139 rows=4808 loops=1)                                             |
| Sort Key: l.nom                                                                                                                          |
| Sort Method: quicksort Memory: 543kB                                                                                                     |
| -> HashAggregate (cost=1657.65..1813.90 rows=62 width=54) (actual time=27.079..28.345 rows=4808 loops=1)                                 |
| Group Key: l.id, l.nom                                                                                                                   |
| Filter: (count(jd.id) = 0)                                                                                                               |
| Batches: 1 Memory Usage: 1425kB                                                                                                          |
| Rows Removed by Filter: 1442                                                                                                             |
| -> Hash Left Join (cost=975.49..1563.90 rows=12500 width=62) (actual time=7.322..21.285 rows=16811 loops=1)                              |
| Hash Cond: (ct.id = jd.id_contrat)                                                                                                       |
| -> Hash Join (cost=893.12..1340.62 rows=12500 width=62) (actual time=6.399..17.963 rows=16657 loops=1)                                   |
| Hash Cond: (l.id_institution = i.id)                                                                                                     |
| -> Hash Right Join (cost=719.62..1134.28 rows=12500 width=58) (actual time=4.894..13.406 rows=16657 loops=1)                             |
| Hash Cond: (p.id_laboratoire_pilote = l.id)                                                                                              |
| -> Hash Right Join (cost=515.00..896.82 rows=12500 width=16) (actual time=3.415..8.124 rows=15411 loops=1)                               |
| Hash Cond: (ct.id_projet = p.id)                                                                                                         |
| -> Seq Scan on contrat ct (cost=0.00..349.00 rows=12500 width=16) (actual time=0.008..0.857 rows=12500 loops=1)                          |
| -> Hash (cost=390.00..390.00 rows=10000 width=16) (actual time=3.349..3.351 rows=10000 loops=1)                                          |
| Buckets: 16384 Batches: 1 Memory Usage: 597kB                                                                                            |
| -> Seq Scan on projet p (cost=0.00..390.00 rows=10000 width=16) (actual time=0.020..1.811 rows=10000 loops=1)                            |
| -> Hash (cost=126.50..126.50 rows=6250 width=50) (actual time=1.465..1.466 rows=6250 loops=1)                                            |
| Buckets: 8192 Batches: 1 Memory Usage: 591kB                                                                                             |
| -> Seq Scan on laboratoire l (cost=0.00..126.50 rows=6250 width=50) (actual time=0.008..0.510 rows=6250 loops=1)                         |
| -> Hash (cost=111.00..111.00 rows=5000 width=20) (actual time=1.492..1.493 rows=5000 loops=1)                                            |
| Buckets: 8192 Batches: 1 Memory Usage: 334kB                                                                                             |
| -> Seq Scan on institution i (cost=0.00..111.00 rows=5000 width=20) (actual time=0.011..0.599 rows=5000 loops=1)                         |
| -> Hash (cost=82.00..82.00 rows=29 width=16) (actual time=0.914..0.915 rows=1956 loops=1)                                                |
| Buckets: 2048 (originally 1024) Batches: 1 (originally 1) Memory Usage: 108kB                                                            |
| -> Bitmap Heap Scan on jeu_donnees jd (cost=4.43..82.00 rows=29 width=16) (actual time=0.149..0.601 rows=1956 loops=1)                   |
| Recheck Cond: (((licence IS NULL) OR (date_depot IS NULL)) AND (EXTRACT(year FROM COALESCE(date_depot, date_creation)) = '2024'::nu...)) |
| Heap Blocks: exact=186                                                                                                                   |
| -> Bitmap Index Scan on idx_jeu_donnees_non_conforme_2024 (cost=0.00..4.42 rows=29 width=0) (actual time=0.116..0.117 rows=1956 lo...)   |
| Planning Time: 0.807 ms                                                                                                                  |
| Execution Time: 37.524 ms                                                                                                                |

FIGURE 10 – Plan d'exécution de la Version 2 de la requête R3 avec index d'optimisation.

## 5.5. Synthèse des optimisations

### 5.5.1 Index créés

```

1  -- Index pour R1
2  CREATE INDEX IF NOT EXISTS idx_jeu_donnees_date_depot ON jeu_donnees
   (date_depot);
3  CREATE INDEX IF NOT EXISTS idx_jeu_donnees_date_creation ON
   jeu_donnees(date_creation);
4  CREATE INDEX IF NOT EXISTS idx_jeu_donnees_contrat ON jeu_donnees(
   id_contrat);
5  CREATE INDEX IF NOT EXISTS idx_contrat_projet ON contrat(id_projet);
6  CREATE INDEX IF NOT EXISTS idx_projet_chercheur_projet ON
   projet_chercheur(id_projet);
7  CREATE INDEX IF NOT EXISTS idx_projet_chercheur_chercheur ON
   projet_chercheur(id_chercheur);
8
9  -- Index pour R2
10 CREATE INDEX IF NOT EXISTS idx_publication_date_publication ON
   publication(date_publication);
11 CREATE INDEX IF NOT EXISTS idx_publication_auteur_chercheur ON
   publication_auteur(id_chercheur);

```

```
12 CREATE INDEX IF NOT EXISTS idx_publication_auteur_publication ON
    publication_auteur(id_publication);
13 CREATE INDEX IF NOT EXISTS idx_chercheur_laboratoire ON chercheur(
    id_laboratoire);
14
15 -- Index pour R3
16 CREATE INDEX IF NOT EXISTS idx_jeu_donnees_non_conforme_2024
17     ON jeu_donnees(id_contrat)
18     WHERE (licence IS NULL OR date_depot IS NULL)
19     AND EXTRACT(YEAR FROM COALESCE(date_depot, date_creation)) =
        2024;
```

Listing 15 – Index créés pour les requêtes R1, R2 et R3

## 6. Partie 5 - Triggers et Procédures Stockées

### 6.1. Triggers

#### 6.1.1 Trigger 1 : Limite de participants à un projet

**Objectif** Vérifier lors de l'insertion d'une participation qu'on ne dépasse pas la capacité maximale du projet.

##### Implémentation

```
1 -- Code SQL du trigger
```

Listing 16 – Trigger limite participants

**Tests** Description des tests effectués et résultats.

#### 6.1.2 Trigger 2 : Vérification DMP d'un jeu de données

**Objectif** Empêcher le passage d'un jeu de données en statut "déposé" si le DMP du contrat n'est pas validé.

##### Implémentation

```
1 -- Code SQL du trigger
```

Listing 17 – Trigger vérification DMP

**Tests** Scénarios de test et résultats.

### 6.2. Procédures stockées et fonctions

#### 6.2.1 Fonction 1 : Nombre de publications d'un projet

**Objectif** Calculer le nombre de publications pour un projet donné et une année donnée.

##### Signature

```
1 CREATE FUNCTION nb_publications_projet(  
2     p_id_projet BIGINT,  
3     p_annee INTEGER  
4 ) RETURNS INTEGER
```

Listing 18 – Signature de la fonction

##### Implémentation

```
1 -- Code SQL complet
```

Listing 19 – Fonction nb\_publications\_projet

**Tests**

```
1 -- Scripts de test
```

Listing 20 – Tests de la fonction

**6.2.2 Procédure 2 : Préparation du bilan d'un projet**

**Objectif** Mettre à jour une table de bilan avec le nombre de publications et jeux de données par projet pour une année donnée.

**Table de bilan**

```
1 -- Structure de la table
```

Listing 21 – Table bilan\_projet

**Implémentation**

```
1 -- Code SQL complet
```

Listing 22 – Procédure preparation\_bilan

**Tests** Scripts de test et résultats.

**6.2.3 Fonction 3 : Fiche projet**

**Objectif** Renvoyer la liste des publications et jeux de données associés à un projet avec leurs principales informations.

**Type de retour** Description de la structure retournée (TABLE, SETOF, JSON, etc.).

**Implémentation**

```
1 -- Code SQL complet
```

Listing 23 – Fonction fiche\_projet

**Tests** Exemples d'utilisation et résultats.

**6.2.4 Procédure 4 : Archivage des contrats échus**

**Objectif** Déplacer les contrats dont la date de fin est antérieure à une date seuil dans une table d'archives.

**Table d'archives**

```
1 -- Structure de la table
```

Listing 24 – Table contrat\_archive

**Implémentation**

```
1 -- Code SQL complet
```

Listing 25 – Procédure archivage\_contrats

**Tests** Scripts de test avec différentes dates seuil.

## **6.3. Documentation du code**

### **6.3.1 Conventions de nommage**

Description des conventions adoptées pour les variables, paramètres, etc.

### **6.3.2 Commentaires**

Exemples de commentaires dans le code PL/pgSQL.

## **7. Conclusion**

### **7.1. Synthèse du travail réalisé**

Récapitulatif des différentes parties et des résultats obtenus.

### **7.2. Apports pédagogiques**

#### **7.2.1 Compétences techniques acquises**

- Conception de schéma relationnel complexe
- Maîtrise de PostgreSQL et PL/pgSQL
- Techniques d'optimisation de requêtes
- Gestion de la sécurité et des accès

#### **7.2.2 Utilisation de l'IA générative**

Retour critique sur l'utilisation des outils d'IA dans le processus de développement.

### **7.3. Difficultés rencontrées**

Liste des principaux défis et solutions apportées.

### **7.4. Perspectives d'amélioration**

Pistes d'amélioration pour la base de données et le processus de développement.

## A. Scripts SQL

### A.1. Script de création du schéma

Référence au fichier `schema_univ_recherche.sql`.

### A.2. Script d'insertion des données

Référence au fichier `inserts_postgres.sql`.

### A.3. Script de gestion des utilisateurs

Référence au fichier `schema_avance.sql` Partie 3.

### A.4. Scripts des requêtes optimisées

Référence au fichier `schema_avance.sql` Partie 4.

### A.5. Scripts des triggers et procédures

Référence au fichier `schema_avance.sql` Partie 5.

## B. Code Python

### B.1. Script de génération de données

Référence au fichier `generate_data.py`.

## C. Captures d'écran

### C.1. Plans d'exécution

Captures d'écran des plans EXPLAIN ANALYZE pour les requêtes optimisées.

### C.2. Tests des procédures

Captures d'écran des résultats des tests.

## D. Dictionnaire de données

Référence au fichier `dictionnaire_attributs.md`.